

Exercise 1: Implementing the Singleton Pattern

The screenshot shows an online Java compiler interface. The code in `Main.java` defines a `Logger` class with a static `log` method and a static `getInstance` method. The `getInstance` method uses a static `instance` variable and a `logCount` to ensure only one instance is created. The `Main` class uses `Logger.getInstance()` to create two logger objects, `logger1` and `logger2`, and prints their equality.

```
15 public void log(String message) {
16     logCount++;
17     System.out.println("[LOG " + logCount + "] " + message);
18 }
19
20 }
21
22 public class Main {
23     public static void main(String[] args) {
24         Logger logger1 = Logger.getInstance();
25         logger1.log("Application started");
26
27         Logger logger2 = Logger.getInstance();
28         logger2.log("User logged in");
29
30         System.out.println("Are logger1 and logger2 the same instance? " + (logger1 == logger2));
31
32     }
33 }
```

Output:

```
Logger instance created.
[LOG 1] Application started
[LOG 2] User logged in
Are logger1 and logger2 the same instance? true
...Program finished with exit code 0
Press ENTER to exit console.
```

The screenshot shows the same online Java compiler interface. The code in `Main.java` defines a `Logger` class with a static `log` method and a static `getInstance` method. The `getInstance` method uses a static `instance` variable and a `logCount` to ensure only one instance is created. The `Main` class uses `Logger.getInstance()` to create two logger objects, `logger1` and `logger2`, and prints their equality.

```
1 class Logger {
2     private static Logger instance;
3     private int logCount = 0;
4
5     private Logger() {
6         System.out.println("Logger instance created.");
7     }
8
9     public static Logger getInstance() {
10        if (instance == null) {
11            instance = new Logger();
12        }
13        return instance;
14    }
15
16    public void log(String message) {
17        logCount++;
18        System.out.println("[LOG " + logCount + "] " + message);
19    }
20 }
```

Output:

```
Logger instance created.
[LOG 1] Application started
[LOG 2] User logged in
Are logger1 and logger2 the same instance? true
...Program finished with exit code 0
Press ENTER to exit console.
```

Exercise 2: Implementing the Factory Method Pattern

onlinegdb.com/online_java_compiler#

```
1 interface Document {
2     void open();
3 }
4
5 class WordDocument implements Document {
6     public void open() {
7         System.out.println("Opening a Word document.");
8     }
9 }
10
11 class PdfDocument implements Document {
12     public void open() {
13         System.out.println("Opening a PDF document.");
14     }
15 }
16
17 class ExcelDocument implements Document {
18     public void open() {
```

input

```
Opening a Word document.
Opening a PDF document.
Opening an Excel document.

...Program finished with exit code 0
Press ENTER to exit console.
```

onlinegdb.com/online_java_compiler#

```
17 class ExcelDocument implements Document {
18     public void open() {
19         System.out.println("Opening an Excel document.");
20     }
21 }
22
23 abstract class DocumentFactory {
24     abstract Document createDocument();
25 }
26
27 class WordDocumentFactory extends DocumentFactory {
28     Document createDocument() {
29         return new WordDocument();
30     }
31 }
32
33 class PdfDocumentFactory extends DocumentFactory {
34     Document createDocument() {
35         return new PdfDocument();
36     }
37 }
38
39 class ExcelDocumentFactory extends DocumentFactory {
40     Document createDocument() {
41         return new ExcelDocument();
42     }
43 }
44
```

input

```
Press ENTER to exit console.
```

ChatGPT x (3961) x Your Re x (5) Wh x (39) x Step-b x Online x Exercis x Digital x + Ask Gemini

onlinegdb.com/online_java_compiler#

YouTube Gmail lonewolf235 (Prakas... FREE AI-Enhanced... GitHub - seshadrimr... Online Java Compil... A Compact and Effic...

Run Debug Stop Share Save Beautify Language Java

Main.java

```
32
33 class PdfDocumentFactory extends DocumentFactory {
34     Document createDocument() {
35         return new PdfDocument();
36     }
37 }
38
39 class ExcelDocumentFactory extends DocumentFactory {
40     Document createDocument() {
41         return new ExcelDocument();
42     }
43 }
```

input

Opening a Word document.
Opening a PDF document.
Opening an Excel document.

...Program finished with exit code 0
Press ENTER to exit console.

28°C Mostly cloudy Search 01:33 11-07-2026

ChatGPT x (3961) x Your Re x (5) Wh x (39) x Step-b x Online x Exercis x Digital x + Ask Gemini

onlinegdb.com/online_java_compiler#

YouTube Gmail lonewolf235 (Prakas... FREE AI-Enhanced... GitHub - seshadrimr... Online Java Compil... A Compact and Effic...

Run Debug Stop Share Save Beautify Language Java

Main.java

```
32
33 class PdfDocumentFactory extends DocumentFactory {
34     Document createDocument() {
35         return new PdfDocument();
36     }
37 }
38
39 class ExcelDocumentFactory extends DocumentFactory {
40     Document createDocument() {
41         return new ExcelDocument();
42     }
43 }
44
45 public class Main {
46     public static void main(String[] args) {
47         DocumentFactory wordFactory = new WordDocumentFactory();
48         Document wordDoc = wordFactory.createDocument();
49         wordDoc.open();
50
51         DocumentFactory pdfFactory = new PdfDocumentFactory();
52         Document pdfDoc = pdfFactory.createDocument();
53         pdfDoc.open();
54
55         DocumentFactory excelFactory = new ExcelDocumentFactory();
56         Document excelDoc = excelFactory.createDocument();
57         excelDoc.open();
58     }
59 }
```

input

Press ENTER to exit console.

28°C Mostly cloudy Search 01:33 11-07-2026

The screenshot shows a web browser with multiple tabs, including 'Ask Gemini'. The active tab is 'onlinegdb.com/online_java_compiler#'. The browser's address bar shows the URL. Below the browser, there is a toolbar with buttons for 'Run', 'Debug', 'Stop', 'Share', 'Save', and 'Beautify'. The main area is a code editor with a dark background, showing a Java file named 'Main.java'. The code defines a base class 'DocumentFactory' and two subclasses: 'PdfDocumentFactory' and 'ExcelDocumentFactory'. Both subclasses implement the 'createDocument()' method. The 'PdfDocumentFactory' returns a new 'PdfDocument()' object, and the 'ExcelDocumentFactory' returns a new 'ExcelDocument()' object. Below the code editor, there is a console window showing the output of the program. The output indicates that the program is opening a Word document, a PDF document, and an Excel document. The program finished with exit code 0, and the user is prompted to press ENTER to exit the console. The bottom of the screen shows a Windows taskbar with a search bar, several application icons, and system information including the temperature (28°C), weather (Mostly cloudy), and time (01:33 on 11-07-2026).

```
32
33 class PdfDocumentFactory extends DocumentFactory {
34     Document createDocument() {
35         return new PdfDocument();
36     }
37 }
38
39 class ExcelDocumentFactory extends DocumentFactory {
40     Document createDocument() {
41         return new ExcelDocument();
42     }
43 }
```

input

Opening a Word document.
Opening a PDF document.
Opening an Excel document.

...Program finished with exit code 0
Press ENTER to exit console.

Exercise 3: Implementing the Builder Pattern

The screenshot shows the same online Java compiler interface as the previous one. The code editor now contains a Java file named 'Main.java' implementing the Builder pattern for a 'Computer' class. The 'Computer' class has three private final attributes: 'cpu', 'ram', and 'storage'. It has a private constructor that takes a 'Builder' object and sets the attributes. There is also a public 'toString()' method that returns a string representation of the computer. A static inner class 'Builder' is defined with three setter methods: 'setCpu()', 'setRam()', and 'setStorage()', each returning the 'Builder' object for method chaining. Finally, there is a public 'build()' method in the 'Computer' class that returns a new 'Computer' object created with the 'Builder'.

```
1 class Computer {
2     private final String cpu;
3     private final String ram;
4     private final String storage;
5
6     private Computer(Builder builder) {
7         this.cpu = builder.cpu;
8         this.ram = builder.ram;
9         this.storage = builder.storage;
10    }
11
12    public String toString() {
13        return "Computer [CPU=" + cpu + ", RAM=" + ram + ", Storage=" + storage + "];"
14    }
15
16    static class Builder {
17        private String cpu;
18        private String ram;
19        private String storage;
20
21        public Builder setCpu(String cpu) {
22            this.cpu = cpu;
23            return this;
24        }
25
26        public Builder setRam(String ram) {
27            this.ram = ram;
28            return this;
29        }
30
31        public Builder setStorage(String storage) {
32            this.storage = storage;
33            return this;
34        }
35
36        public Computer build() {
37            return new Computer(this);
38        }
39    }
40 }
```

The screenshot shows a web browser with multiple tabs open, including ChatGPT, YouTube, Gmail, and various social media links. The active tab is 'onlinegdb.com/online_java_compiler#'. The browser's address bar shows the URL. Below the browser, there is a taskbar with various application icons and a system tray showing the temperature (28°C), weather (Mostly cloudy), and time (01:41, 11-07-2026).

```
22 public Builder setProcessor(String cpu) {
23     this.cpu = cpu;
24     return this;
25 }
26
27 public Builder setRam(String ram) {
28     this.ram = ram;
29     return this;
30 }
31
32 public Builder setStorage(String storage) {
33     this.storage = storage;
34     return this;
35 }
36
37 public Computer build() {
38     return new Computer(this);
39 }
40 }
41
42 public class Main {
43     public static void main(String[] args) {
44         Computer gamingPC = new Computer.Builder()
45             .setCpu("Intel i9")
46             .setRam("32GB")
47             .setStorage("2TB SSD")
48             .build();
49
50         Computer officePC = new Computer.Builder()
51             .setCpu("Intel i5")
52             .setRam("8GB")
53             .setStorage("512GB SSD")
54             .build();
55
56         System.out.println(gamingPC);
57         System.out.println(officePC);
58     }
59 }
```

The screenshot shows the same web browser and taskbar as the first image. The output of the Java program is displayed in the console area below the code editor. The output shows the details of the two computers created: a gaming PC with Intel i9, 32GB RAM, and 2TB SSD, and an office PC with Intel i5, 8GB RAM, and 512GB SSD. The program finished with exit code 0.

```
Computer [CPU=Intel i9, RAM=32GB, Storage=2TB SSD]
Computer [CPU=Intel i5, RAM=8GB, Storage=512GB SSD]

...Program finished with exit code 0
Press ENTER to exit console.
```

Exercise 4: Adapter Pattern

```
1- Interface PaymentProcessor {
2-     void processPayment(double amount);
3- }
4-
5- class PayPalGateway {
6-     void makePayment(double amountInUsd) {
7-         System.out.println("PayPal: processed payment of $" + amountInUsd);
8-     }
9- }
10-
11- class StripeGateway {
12-     void charge(double amountInUsd) {
13-         System.out.println("Stripe: charged $" + amountInUsd);
14-     }
15- }
16-
17- class PayPalAdapter implements PaymentProcessor {
18-     private PayPalGateway paypalGateway = new PayPalGateway();
19-
20-     public void processPayment(double amount) {
21-         paypalGateway.makePayment(amount);
22-     }
23- }
24-
25- class StripeAdapter implements PaymentProcessor {
26-     private StripeGateway stripeGateway = new StripeGateway();
27-
28-     public void processPayment(double amount) {
29-         stripeGateway.charge(amount);
30-     }
31- }
32-
33- public class Main {
```

Input

...Program finished with exit code 0
Press ENTER to exit console.

```
20-     public void processPayment(double amount) {
21-         paypalGateway.makePayment(amount);
22-     }
23- }
24-
25- class StripeAdapter implements PaymentProcessor {
26-     private StripeGateway stripeGateway = new StripeGateway();
27-
28-     public void processPayment(double amount) {
29-         stripeGateway.charge(amount);
30-     }
31- }
32-
33- public class Main {
34-     public static void main(String[] args) {
35-         PaymentProcessor paypal = new PayPalAdapter();
36-         paypal.processPayment(100.0);
37-
38-         PaymentProcessor stripe = new StripeAdapter();
39-         stripe.processPayment(250.0);
40-     }
41- }
```

Input

PayPal: processed payment of \$100.0
Stripe: charged \$250.0

...Program finished with exit code 0
Press ENTER to exit console.

Exercise 5: Decorator Pattern

The screenshot shows an online Java compiler interface with the following code in `Main.java`:

```
1- interface Notifier {
2-     void send(String message);
3- }
4-
5- class EmailNotifier implements Notifier {
6-     public void send(String message) {
7-         System.out.println("Sending EMAIL: " + message);
8-     }
9- }
10-
11- abstract class NotifierDecorator implements Notifier {
12-     protected Notifier wrapped;
13-
14-     NotifierDecorator(Notifier wrapped) {
15-         this.wrapped = wrapped;
16-     }
17-
18-     public void send(String message) {
19-         wrapped.send(message);
20-     }
21- }
22-
23- class SMSNotifierDecorator extends NotifierDecorator {
24-     SMSNotifierDecorator(Notifier wrapped) {
25-         super(wrapped);
26-     }
27-
28-     public void send(String message) {
29-         super.send(message);
30-         System.out.println("Sending SMS: " + message);
31-     }
32- }
```

The console output shows:

```
Sending SMS: Server is down!
Sending SLACK message: Server is down!
...Program finished with exit code 0
Press ENTER to exit console.
```

The screenshot shows an online Java compiler interface with the following code in `Main.java`:

```
26- }
27-
28- public void send(String message) {
29-     super.send(message);
30-     System.out.println("Sending SMS: " + message);
31- }
32- }
33-
34- class SlackNotifierDecorator extends NotifierDecorator {
35-     SlackNotifierDecorator(Notifier wrapped) {
36-         super(wrapped);
37-     }
38-
39-     public void send(String message) {
40-         super.send(message);
41-         System.out.println("Sending SLACK message: " + message);
42-     }
43- }
44-
45- public class Main {
46-     public static void main(String[] args) {
47-         Notifier notifier = new SlackNotifierDecorator(
48-             new SMSNotifierDecorator(
49-                 new EmailNotifier());
50-         );
51-         notifier.send("Server is down!");
52-     }
53- }
```

The console output shows:

```
Sending EMAIL: Server is down!
Sending SMS: Server is down!
Sending SLACK message: Server is down!
...Program finished with exit code 0
Press ENTER to exit console.
```

Exercise 6: Proxy Pattern

onlinegdb.com/online_java_compiler#

```
1- Interface Image {
2-     void display();
3- }
4-
5- class RealImage implements Image {
6-     private String filename;
7-
8-     RealImage(String filename) {
9-         this.filename = filename;
10-        loadFromServer();
11-    }
12-
13-    private void loadFromServer() {
14-        System.out.println("Loading " + filename + " from remote server...");
15-    }
16-
17-    public void display() {
18-        System.out.println("Displaying " + filename);
19-    }
20- }
21-
22- class ProxyImage implements Image {
23-     private RealImage realImage;
24-     private String filename;
25-
26-     ProxyImage(String filename) {
27-         this.filename = filename;
28-     }
29-
30-    public void display() {
31-        if (realImage == null) {
32-            realImage = new RealImage(filename);
33-        }
34-        realImage.display();
35-    }
36- }
37-
38- public class Main {
```

28°C Mostly cloudy 01:50 11-07-2026

onlinegdb.com/online_java_compiler#

```
27-     this.filename = filename;
28- }
29-
30- public void display() {
31-     if (realImage == null) {
32-         realImage = new RealImage(filename);
33-     }
34-     realImage.display();
35- }
36- }
37-
38- public class Main {
39-     public static void main(String[] args) {
40-         Image image = new ProxyImage("photo.jpg");
41-         System.out.println("First call:");
42-         image.display();
43-         System.out.println("\nSecond call (should not reload):");
44-         image.display();
45-     }
46- }
47-
48- }
```

Input

```
First call:
Loading photo.jpg from remote server...
Displaying photo.jpg

Second call (should not reload):
Displaying photo.jpg

...Program finished with exit code 0
Press ENTER to exit console.
```

28°C Mostly cloudy 01:50 11-07-2026